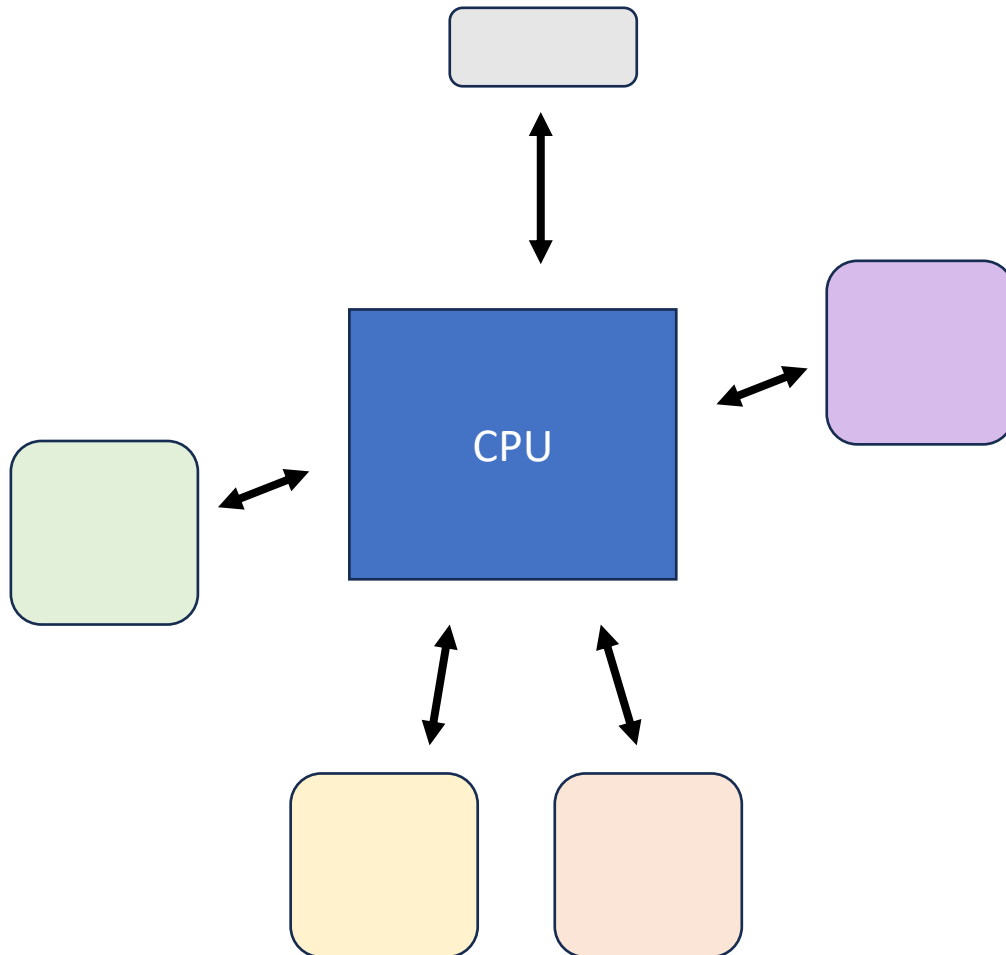


Devices

ECEN 427
Jeff Goeders

BYU Electrical & Computer
Engineering
IRA A. FULTON COLLEGE OF ENGINEERING

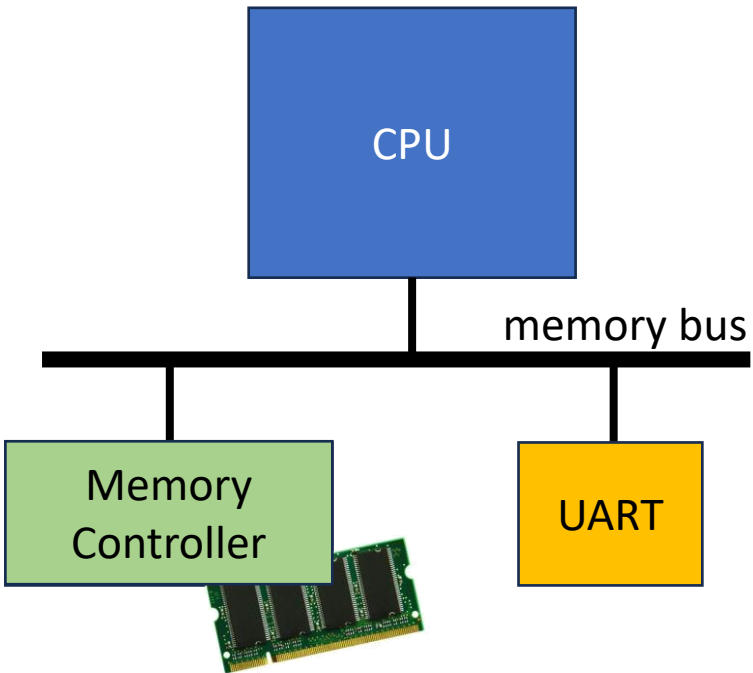
How does a CPU talk to other stuff?



- You learned about various CPU instructions in EC EN 224
 - Arithmetic (add/sub/mul)
 - Control (call/ret/jmp)
 - Data Movement (mov, push, pop)
- In x86 **mov** moves data between:
 - Register → Register
 - Immediate → Register / Memory
 - Memory → Register **Load / Read**
 - Register → Memory **Store / Write**
- Data movement to/from memory addresses is how a CPU talks to devices

Many other instruction sets (e.g. ARM) have dedicated operations for load/store

Load / Store Memory Operations



- **load** (read) and **store** (write) CPU operations trigger a **bus transaction**
 - The CPU hardware is built to do this
 - We will learn more about busses later in the course, and you will design your own hardware on the bus.
- When does **load** and **store** happen in our C code?
 - Accessing variables (sometimes; compiler may map our variables to registers or memory)
 - Dereferencing pointers (*usually* – see next slide)
- Memory operations are not just to access memory; the CPU uses them to communicate with all devices on the bus
- In fact, even when accessing memory, we are really communicating with the **memory controller**, which in turn interfaces with the actual memory device.

The Volatile Keyword

The compiler is allowed to optimize away memory accesses if it can prove the value is already known:

```
int x = *ptr;  
int y = *ptr; // compiler may reuse the loaded value in a register  
              // – only one actual load instruction generated
```

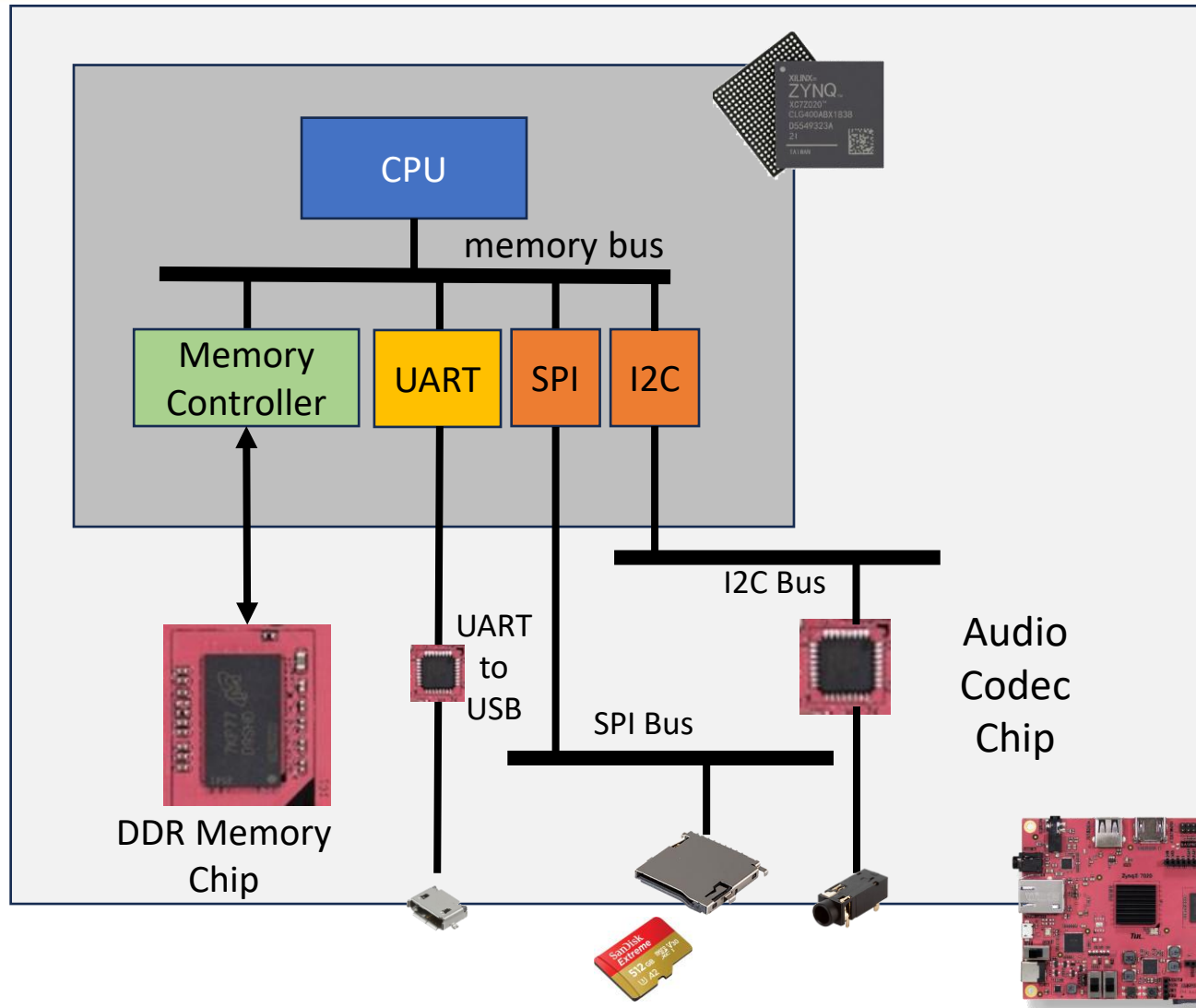
Or if the result is never used:

```
*ptr = 5;  
*ptr = 10; // first store may be eliminated entirely
```

volatile tells the compiler: "don't optimize this away, always really access memory":

```
volatile int *ptr = ...;  
int x = *ptr;           // guaranteed load every time  
*ptr = 5;               // guaranteed store every time
```

Context: On-Chip / Off-Chip Busses

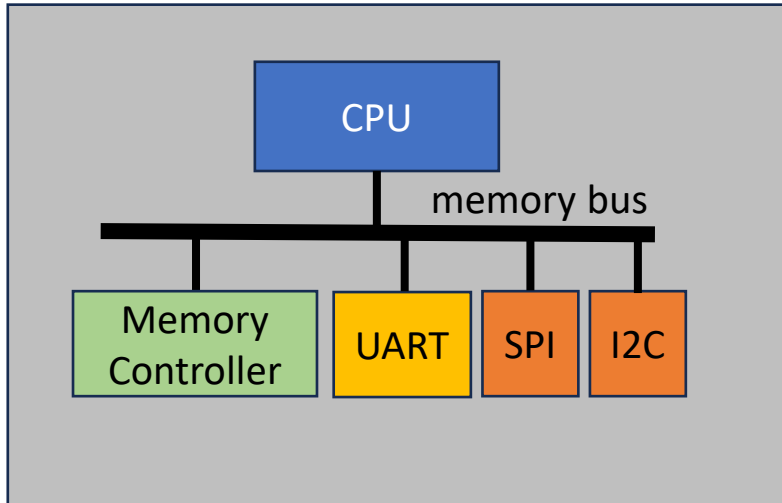


The memory bus is contained **on-chip**;
designed for **intra-chip** communication.
(for most modern devices)

There are other bus protocols designs for
inter-chip communication (e.g. I2C, SPI)

- Longer distances, between chips on a PCB
- Typically slower than the memory bus

The Address Map



Address space refers to the full set of addresses available to the CPU.

- Our CPU has 32-bit addresses, so a $2^{32} = 4\text{GB}$ address space.

The **address map** lists how the address space is split up and assigned to devices on the memory bus.

Loads/stores will be routed to the appropriate device based on this mapping.

Address	Size	Device
0x0000_0000	512 MB	DDR memory
0x4121_0000	64 KB	RGB LEDs GPIO
0x4122_0000	64 KB	Buttons GPIO
0x4123_0000	64 KB	Switches GPIO
0x4124_0000	64 KB	LEDs GPIO
0x4180_0000	64 KB	Interrupt controller
...	...	...

Important notes:

- **System memory is part of this address space** (DDR for our board). In some embedded systems you might see multiple memories.
 - Eg. Microcontroller that uses addresses in flash for the program, and addresses in SRAM for data.
- **These are physical addresses.** We can use pointers to access them in a bare metal system, but not in system with virtual memory.
- **The address space is not full.** There are many “gaps”. If you try to perform a load/store on an address with no device at the other end, results may range from exception to full system hang.

**When you write to memory,
you're not writing to memory.**

*You're sending a message to a device on a bus.
What happens next is up to the device.*

The Illusion of Memory

In C, this looks like writing to RAM:

```
*ptr = 0x1;
```

But the CPU doesn't know what's there.

- It puts the address and value on the bus and moves on.
- The device on the other end decides what to do with it.

Devices Can Do Anything. A write might:

- Store a value (RAM — the boring case)
- Trigger a hardware reset
- Start a DMA transfer
- Toggle a GPIO pin
- Clear an interrupt
- Be silently ignored

The load/store is a message to the device


```
*ptr = 0xFF;  
int x = *ptr; // x might not be 0xFF
```

A device is free to return whatever it wants on a read:

- A status register returns current hardware state
- A FIFO register returns the next item in a queue
- A counter register returns a value that changes every read
- A write-only register returns 0 (or garbage)

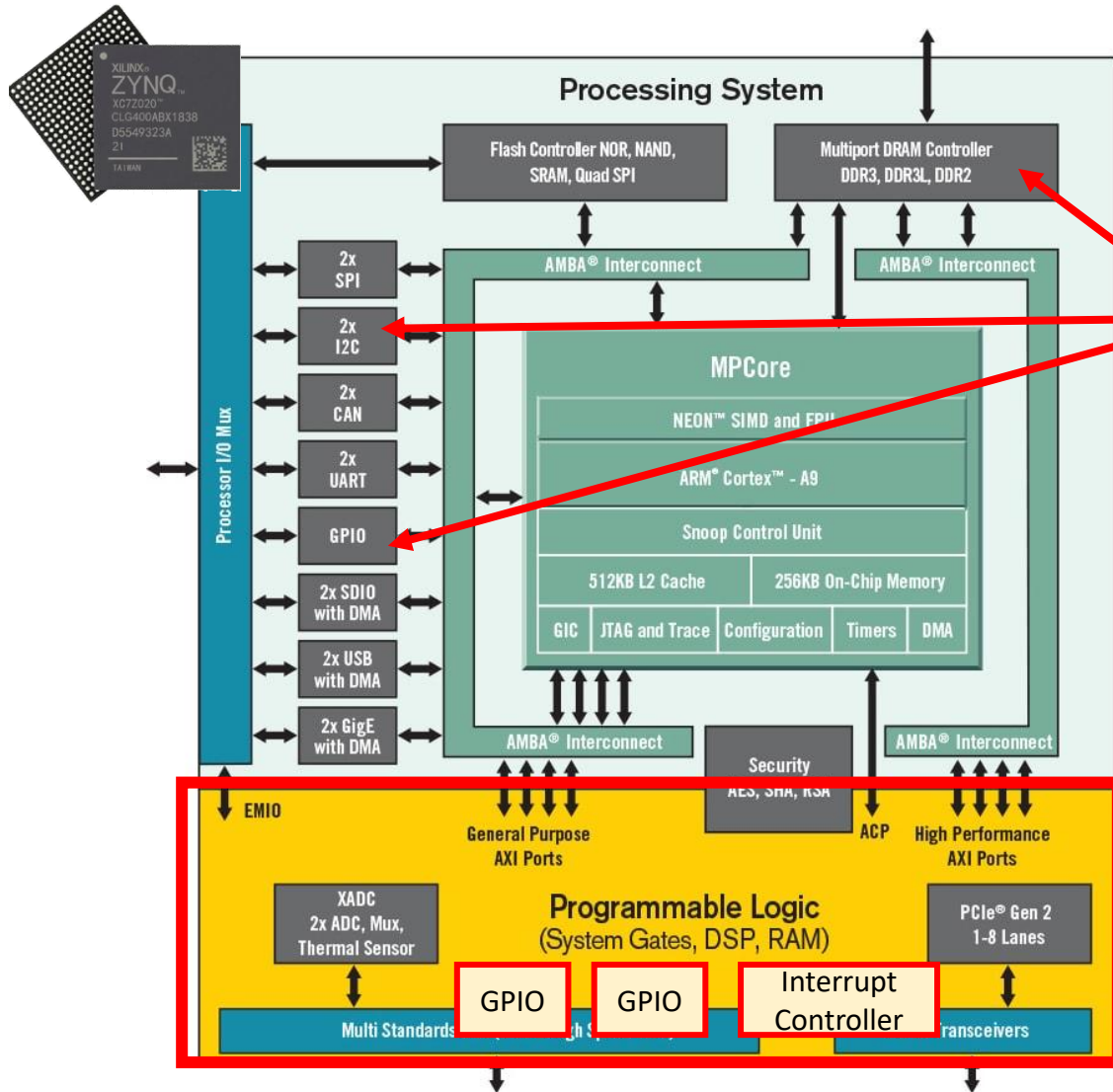
Memory-mapped I/O is a conversation, not a variable.

```
*ptr = 0xFF;  
int y = *ptr; // compiler: "I just wrote 0xFF, y must be 0xFF"  
              // → optimizes away the read entirely
```

This is **wrong** for hardware registers.

`volatile` tells the compiler: *"don't assume anything — always go to the bus."*

Zynq Chip w/ ARM A9 CPU



- This is the architecture diagram for the Zynq-7000 chip on the PYNQ Z-2 board
- The CPU has several “hard” blocks it interfaces with
 - We often call these “IP” (Intellectual Property)
- This is the FPGA part of the chip
 - There are more “IP blocks” implemented in the FPGA bitstream given to you
 - Your system programs this bitstream as part of bootup

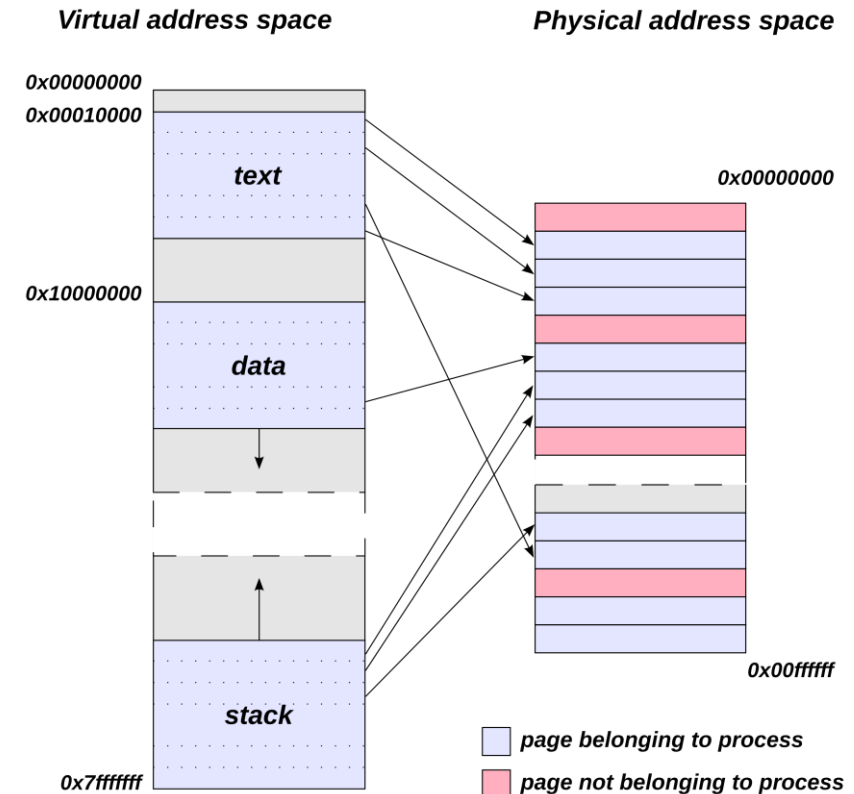
<https://byu-cpe.github.io/ecen427/documentation/hardware/>

Talking to Devices with the Linux OS

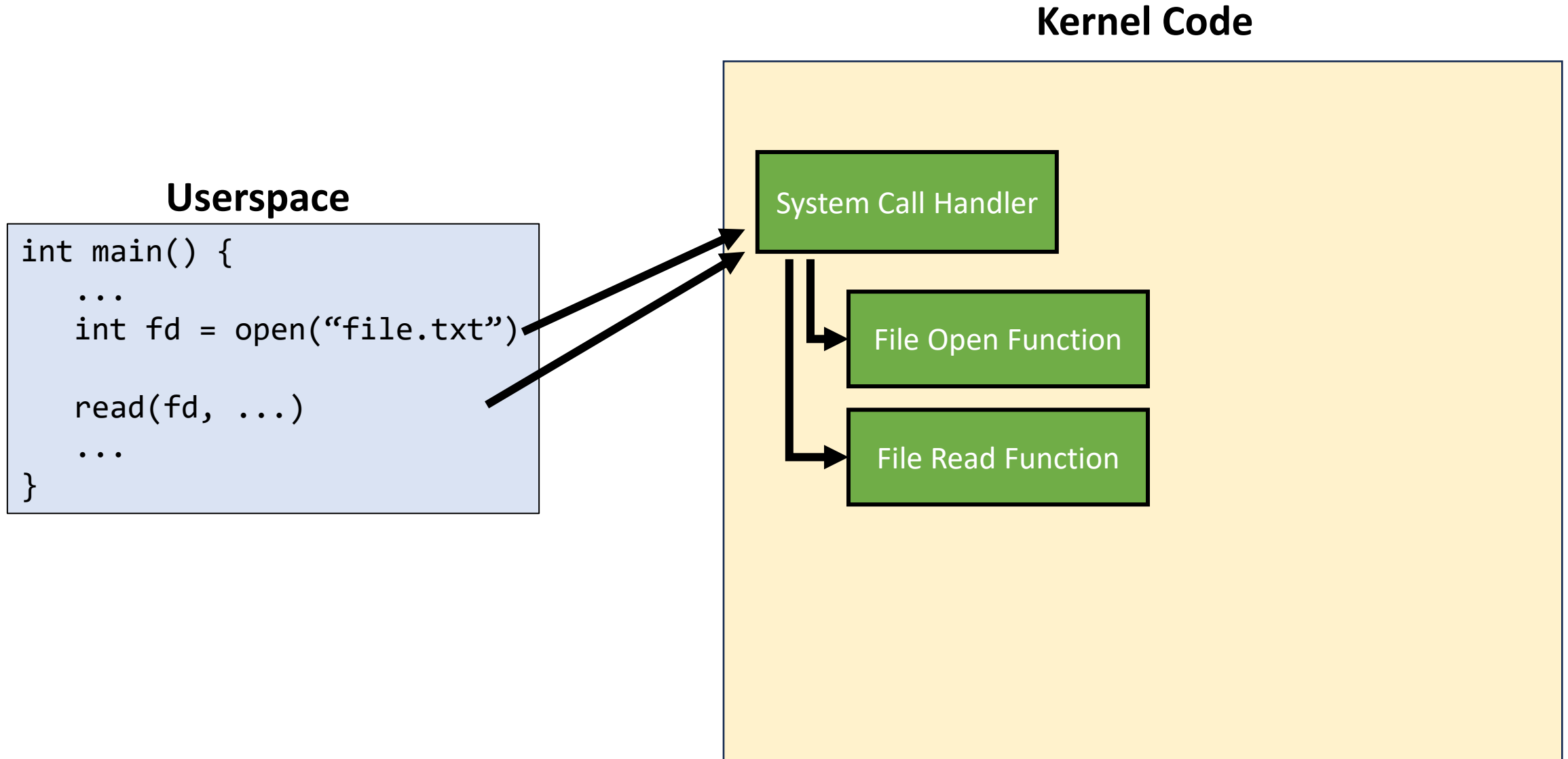
Devices & User Space

- **Recall:** From user space, a process can only talk to physical addresses mapped to a virtual address
- **Problem:** By default, device addresses are not mapped to the user virtual address space.
- **Solution:** We need the help of code within the Kernel to interface with hardware (i.e. kernel drivers)

Remember, system calls are how we communicate with the kernel from user space.



Linux System Calls



At the end of last lecture we looked at the list of syscalls:

- <https://chromium.googlesource.com/chromiumos/docs/+/HEAD/constants/syscalls.md>

Where are the system calls to talk to hardware devices?

- UART
- Disk
- Video card
- Physical memory
- USB devices

Calling Function in Drivers

Userspace

```
int main() {  
    ...  
    uart_get_input() ???  
    ...  
}
```

Don't our device drivers provide functions that we can call to access the devices?

- Look like ordinary files in the filesystem, but are special files
- They are **interfaces to the driver** that controls the device
- By using system calls that operate on files, our code can interface with the driver.
- They are not real files...

Have you used a device file before?

You used `/dev/ttyUSB1` to talk to the serial driver during Lab 1.

...an aside...pseudo-devices

Pseudo-devices [\[edit\]](#)

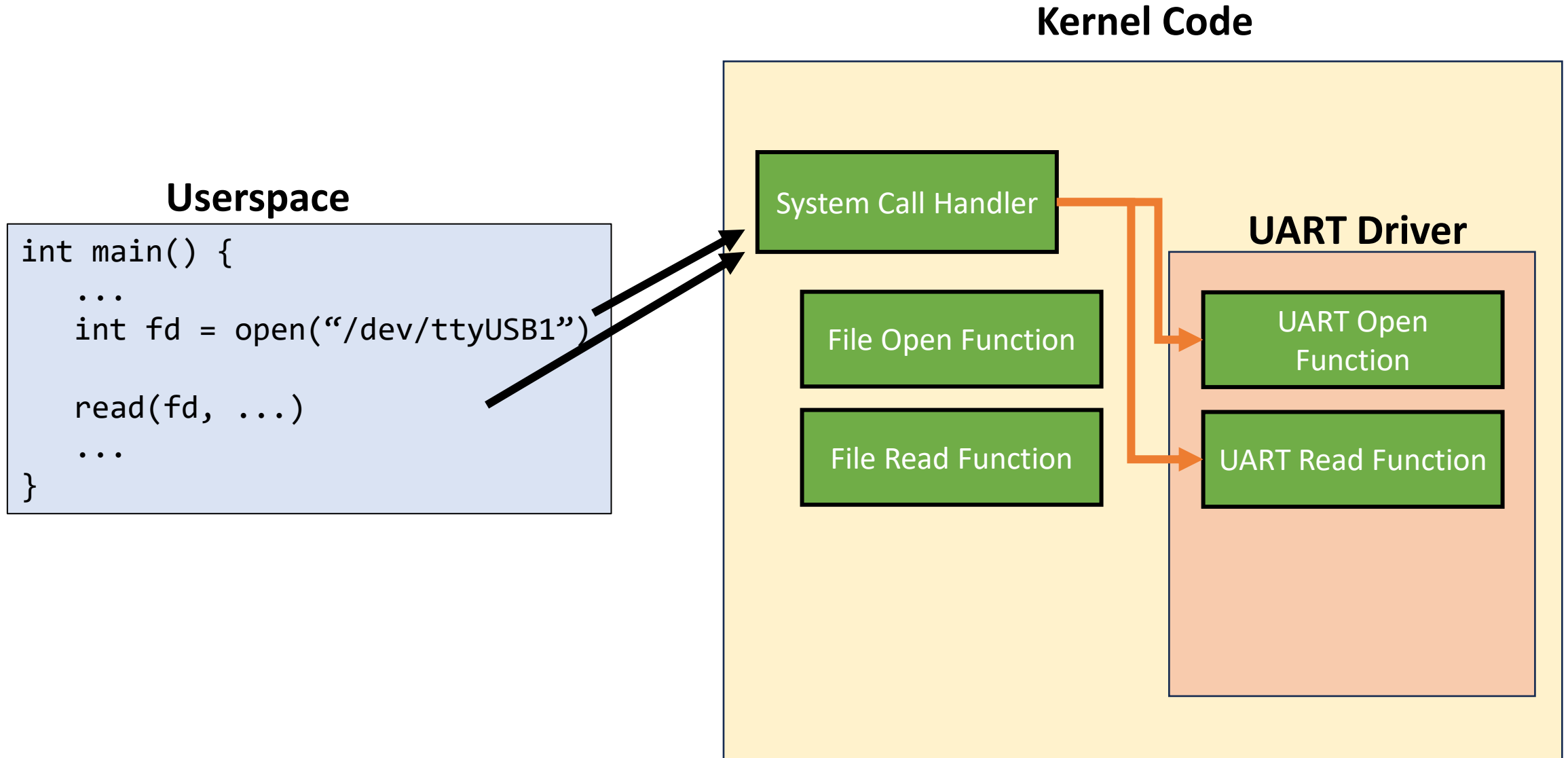
Device nodes on Unix-like systems do not necessarily have to correspond to [physical devices](#). Nodes that lack this correspondence form the group of *pseudo-devices*. They provide various functions handled by the operating system. Some of the most commonly used (character-based) pseudo-devices include:

- [/dev/null](#) – accepts and discards all input written to it; provides an [end-of-file](#) indication when read from.
- [/dev/zero](#) – accepts and discards all input written to it; produces a continuous stream of [null characters](#) (zero-value bytes) as output when read from.
- [/dev/full](#) – produces a continuous stream of null characters (zero-value bytes) as output when read from, and generates an [ENOSPC](#) ("disk full") error when attempting to write to it.
- [/dev/random](#) – produces bytes generated by the kernel's [cryptographically secure pseudorandom number generator](#). Its exact behavior varies by implementation, and sometimes variants such as [/dev/urandom](#) or [/dev/arandom](#) are also provided.
- [/dev/stdin](#), [/dev/stdout](#), [/dev/stderr](#) – access the process's [standard streams](#).
- [/dev/fd/n](#) – accesses the process's [file descriptor](#) *n*.

Additionally, BSD-specific pseudo-devices with an [ioctl](#) interface may also include:

- [/dev/pf](#) – allows userland processes to control [PF](#) through an [ioctl](#) interface.
- [/dev/bio](#) – provides [ioctl](#) access to devices otherwise not found as [/dev](#) nodes, used by [bioctl](#) to implement [RAID](#) management in [OpenBSD](#) and [NetBSD](#).
- [/dev/sysmon](#) – used by NetBSD's [envsys](#) framework for [hardware monitoring](#), accessed in the userland through [proplib\(3\)](#) by the [envstat](#) utility.^[8]

Linux System Calls



ECEN427 PYNQ Hardware System

- <https://byu-cpe.github.io/ecen427/documentation/hardware/>

Linux System Calls

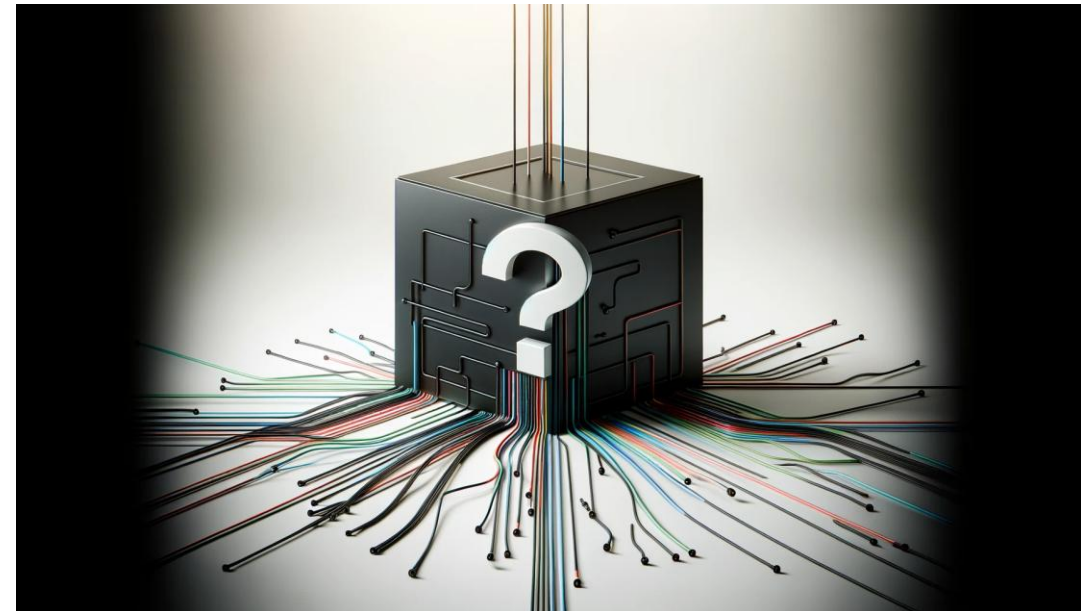
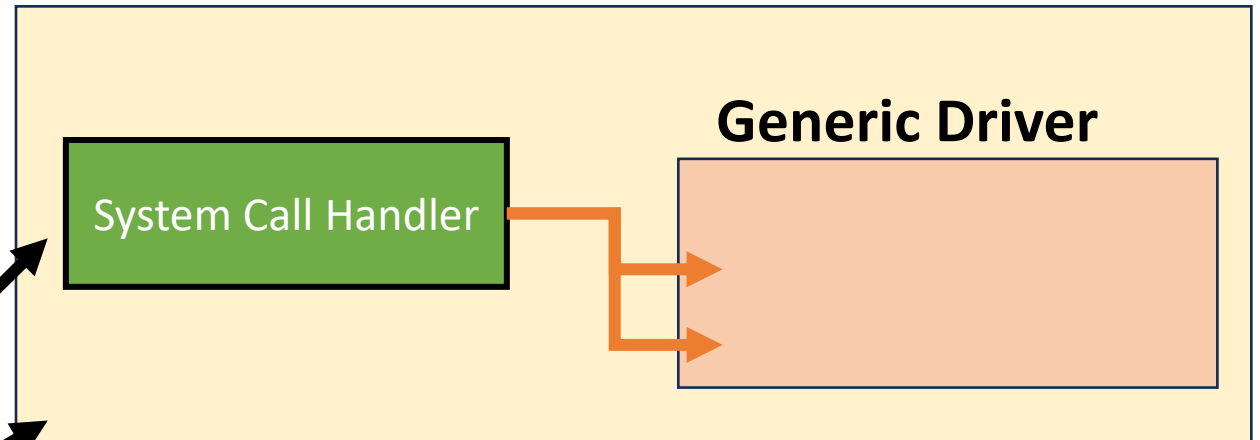
Do we really need a separate driver for every hardware device?

Can we make a generic driver?

Userspace

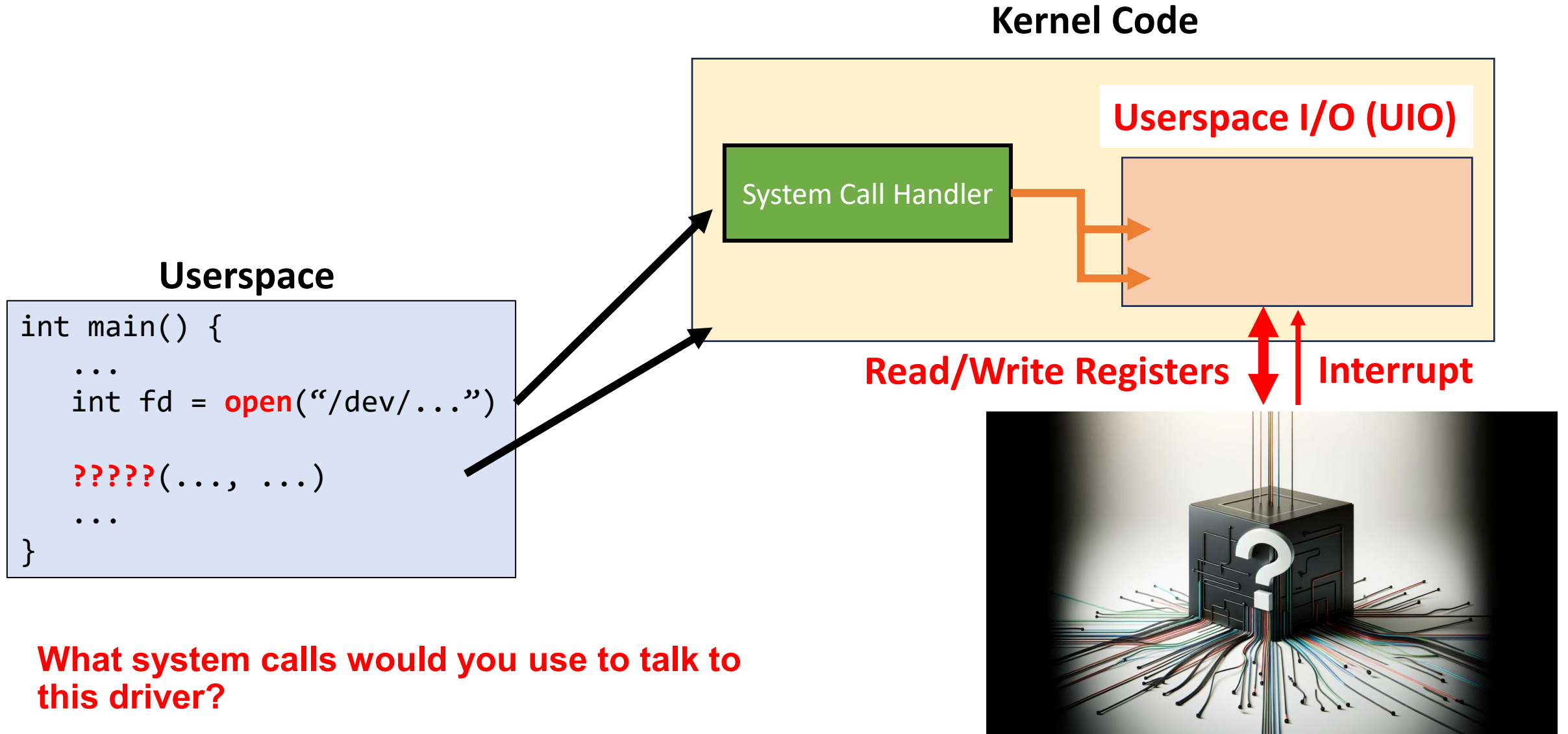
```
int main() {  
    ...  
    int fp = open("/dev/...")  
    ?????(..., ...)  
    ...  
}
```

Kernel Code



What kinds of things do you need to do with a hardware device you know nothing about?

Linux System Calls



What system calls would you use to talk to this driver?

Before learning more, let's pause and
cover a few more operating system
concepts...

`mmap()`

Map files (or devices) into memory

Process States

- Last lecture we talked briefly about processes, and how the operating system virtualizes the CPU in order to run multiple processes.
- The OS scheduler is responsible for selecting which process will run in the next **time slice**.
- Sometimes processes “block” and need to wait for an OS operation to complete.
 - Example: process calls `read()` to get data from a file on disk. This can take several milliseconds.
 - Rather than blocking all execution, the process goes to sleep, and the OS can wake it back up when the data is ready.

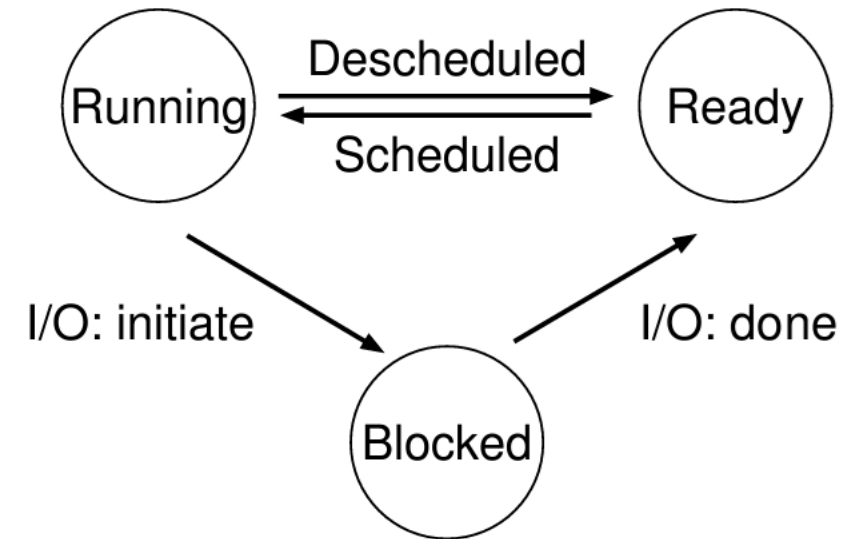


Figure 4.2: **Process: State Transitions**

The Linux Userspace I/O (UIO) driver, provides **generic** access to hardware devices allowing userspace to:

1. Access device memory addresses (read/write regs)
2. Check if device generated an interrupt

- **Read/Write addresses (registers):**
 - `mmap()`
 - Access registers via pointer returned from `mmap`
- **Enable interrupts:**
 - `write()` a value of 1
 - *You must write 4 bytes*
- **Block/wait for interrupt :**
 - `read()`
 - *You must read 4 bytes*
- **Check for interrupt (non-blocking):**
 - `poll()`
 - Used to check if file has data to read

ECEN427 PYNQ Software System

- <https://byu-cpe.github.io/ecen427/documentation/software-stack/>

- The UIO linux driver is not meant to actually serve as the device driver
- It provides an interface that allows you to write drivers in userspace.
- In lab 2, you will create userspace drivers for the buttons, switches and interrupt controller
 - These will interface with the UIO kernel driver to access the devices
- Example https://github.com/byu-cpe/ecen427_student/blob/main/userspace/drivers/uio_example/generic_uio_example.c